

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра программирования и информационных технологий

**Разработка системы онлайн-заказа еды и административного
интерфейса YummiBox**

Бакалаврская работа
09.03.04 Программная инженерия
Профиль Информационные системы и сетевые технологии

Зав. кафедрой _____ С. Д. Махортов д.ф.-м.н., доцент _____.20____

Обучающийся _____ М. М. З. Насайр

Руководитель _____ А. А. Вахтин к.ф.-м.н., доцент

Воронеж 2025

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

УТВЕРЖДАЮ
заведующий кафедрой
программирования и
информационных технологий

_____ С. Д. Махортов
____.____.2025

ЗАДАНИЕ
НА ВЫПОЛНЕНИЕ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
ОБУЧАЮЩЕГОСЯ Фамилия Имя Отчество (в родительном падеже)

1. Тема работы «Название темы выпускной квалификационной работы», утверждена решением ученого совета факультета компьютерных наук от 23.10.2024
2. Направление подготовки: 09.03.04 Программная инженерия
3. Срок сдачи законченной работы: __.__.2025
4. Календарный план (строится в соответствии со структурой ВКР):

№	Структура ВКР	Сроки выполнения	Примечание
1	Введение	01.10.2024 – 07.10.2024	
2	1. Аналитический раздел 1	07.10.2024 – 10.11.2024	
3	1.1 Анализ существующих решений 1.1	07.10.2024 – 14.10.2024	
4	1.2 Выбор архитектуры приложения 1.2	15.10.2024 – 18.10.2024	
5	1.3 Постановка задачи 1.3	19.10.2024 – 22.10.2024	
6	1.4 Обоснование выбора инструментов 1.4	23.10.2024 – 26.10.2024	
7	2. Проектирование системы 2	11.11.2024 – 20.12.2024	
8	2.1 Структура пользовательского приложения 2.1	11.11.2024 – 13.11.2024	
9	2.2 Структура административной панели 2.2	14.11.2024 – 16.11.2024	
10	2.3 Модель базы данных Firebase 2.3	17.11.2024 – 20.11.2024	
11	2.4 Диаграмма вариантов использования (Use Case) 2.4	21.11.2024 – 23.11.2024	
12	2.5 Диаграмма потоков данных (DFD) 2.5	24.11.2024 – 26.11.2024	
13	2.6 Диаграмма классов (Class Diagram) 2.6	27.11.2024 – 29.11.2024	
14	2.7 Диаграмма последовательности (Sequence Diagram) 2.7	30.11.2024 – 02.12.2024	
15	2.8 Диаграмма состояний (State Diagram) 2.8	03.12.2024 – 05.12.2024	
16	2.9 Диаграмма компонентов системы 2.9	06.12.2024 – 08.12.2024	
17	2.10 Диаграммы потоков данных 2.10	09.12.2024 – 11.12.2024	

18	2.11 Прототипы интерфейсов 2.11	12.12.2024 – 14.12.2024	
19	3. Реализация проекта 3	21.12.2024 – 31.01.2025	
20	3.1 Реализация клиентского мобильного приложения 3.1	21.12.2024 – 05.01.2025	
21	3.2 Реализация административной панели 3.2	06.01.2024 – 12.01.2025	
22	3.3 Интеграция с Firebase и Supabase 3.3	13.01.2024 – 17.01.2025	
23	3.4 Авторизация и регистрация пользователей 3.4	18.01.2024 – 20.01.2025	
24	3.5 Загрузка и отображение изображений (Glide + Supabase) 3.5	21.01.2024 – 24.01.2025	
25	4. Анализ ключевых фрагментов кода 4	01.02.2025 – 15.02.2025	
26	5. Тестирование и отладка 5	16.02.2025 – 10.03.2025	
27	5.1 Методика тестирования 5.1	16.02.2025 – 20.02.2025	
28	5.2 Тест-кейсы и результаты 5.2	21.02.2025 – 27.02.2025	
29	5.3 Обнаруженные ошибки и способы их устранения 5.3	28.02.2025 – 10.03.2025	
30	6. Экономическая часть 6	11.03.2025 – 25.03.2025	
31	6.1 Расчёт затрат на разработку 6.1	11.03.2025 – 17.03.2025	
32	6.2 Оценка экономической эффективности 6.2	18.03.2025 – 25.03.2025	
33	Заключение	26.03.2025 – 30.03.2025	
34	Список использованных источников	31.03.2025 – 02.04.2025	
35	Приложения	03.04.2025 – 30.05.2025	

Обучающийся

Насайр М. М. З.

Руководитель

Вахтин А. А.

РЕФЕРАТ

Бакалаврская работа 54 с., 07 рис., 02 табл., 08 использованных источников, 02 приложений.

МОБИЛЬНОЕ ПРИЛОЖЕНИЕ, ДОСТАВКА ЕДЫ, FLUTTER, FIREBASE, SUPABASE, ANDROID, УВЕДОМЛЕНИЯ, ЗАКАЗ, АДМИНИСТРИРОВАНИЕ

Объект исследования – информационная система для онлайн-заказа и обработки еды.

Цель исследования – разработка мобильного клиентского и административного приложений для автоматизации процесса заказа и управления доставкой еды с использованием современных облачных технологий.

Результаты работы: Разработано кроссплатформенное приложение YummiBox (клиентская и административная части) с интеграцией Firebase (Auth, Firestore, FCM) и Supabase (Storage). Построена архитектура приложения, реализована обработка заказов и уведомлений, проведено тестирование на реальных устройствах. Система полностью готова к использованию и масштабированию.

Область применения результатов: Автоматизация заказов в кафе, ресторанах, службах доставки еды и других предприятиях общественного питания малого и среднего бизнеса.

Содержание

Введение.....	7
1 Аналитический раздел	9
1.1 Анализ существующих решений	9
1.2 Выбор архитектуры приложения	13
1.3 Постановка задачи	13
1.4 Обоснование выбора инструментов	13
2 Проектирование системы	15
2.1 Структура пользовательского приложения	15
2.2 Структура административной панели.....	15
2.3 Модель базы данных Firebase	16
2.4 Диаграмма вариантов использования (Use Case)	19
2.5 Диаграмма потоков данных (DFD).....	21
2.6 Диаграмма классов (Class Diagram).....	23
2.7 Диаграмма последовательности (Sequence Diagram)	25
2.8 Диаграмма состояний (State Diagram)	26
2.9 Диаграмма компонентов системы.....	27
2.10 Диаграммы потоков данных	27
2.11 Прототипы интерфейсов.....	27
3 Реализация проекта	29
3.1 Реализация клиентского мобильного приложения.....	29
3.2 Реализация административной панели	29
3.3 Интеграция с Firebase и Supabase	30
3.4 Авторизация и регистрация пользователей.....	31
3.5 Загрузка и отображение изображений (Glide + Supabase).....	31
4 Анализ ключевых фрагментов кода.....	33
5 Тестирование и отладка	44
5.1 Методика тестирования	44
5.2 Тест-кейсы и результаты	44

5.3	Обнаруженные ошибки и способы их устранения	45
6	Экономическая часть	46
6.1	Расчёт затрат на разработку	46
6.2	Оценка экономической эффективности	46
	Заключение.....	47
7	Список использованных источников	48
8	Приложения	49
8.1	Скриншоты интерфейса	49

Введение

1 Актуальность темы

Современные пользователи всё чаще обращаются к мобильным сервисам доставки еды. Повышенный спрос на подобные приложения требует не только удобного пользовательского интерфейса, но и эффективной административной системы управления. Поэтому разработка комплексного мобильного решения — одновременно для клиента и администратора — является актуальной задачей в области информационных технологий и цифровой коммерции.

2 Цель и задачи проекта

Целью проекта является создание мобильного приложения YummiBox с двумя функциональными частями: пользовательской и административной.

Задачи проекта включают:

- Разработка клиентской части приложения на Android (Kotlin);
- Реализация функциональности регистрации, авторизации, просмотра меню и оформления заказа;
- Создание административной панели для управления категориями, товарами, заказами и пользователями;
- Интеграция с облачными сервисами Firebase и Supabase для хранения данных и файлов;
- Обеспечение безопасности и стабильности взаимодействия клиента с сервером.

3 Объект и предмет исследования

Объектом исследования является процесс взаимодействия пользователей с мобильными сервисами доставки еды.

Предметом исследования является разработка мобильного приложения и административной панели с использованием современных облачных и мобильных технологий.

4 Используемые технологии

В ходе разработки использовались следующие технологии и инструменты:

- **Android Studio, Kotlin** — для создания мобильного приложения;
- **Firebase Realtime Database** — для хранения и синхронизации данных в реальном времени;
- **Firebase Authentication** — для реализации безопасной аутентификации пользователей;
- **Supabase Storage** — для хранения изображений и медиафайлов;
- **Glide** — библиотека для быстрой загрузки изображений в приложении.

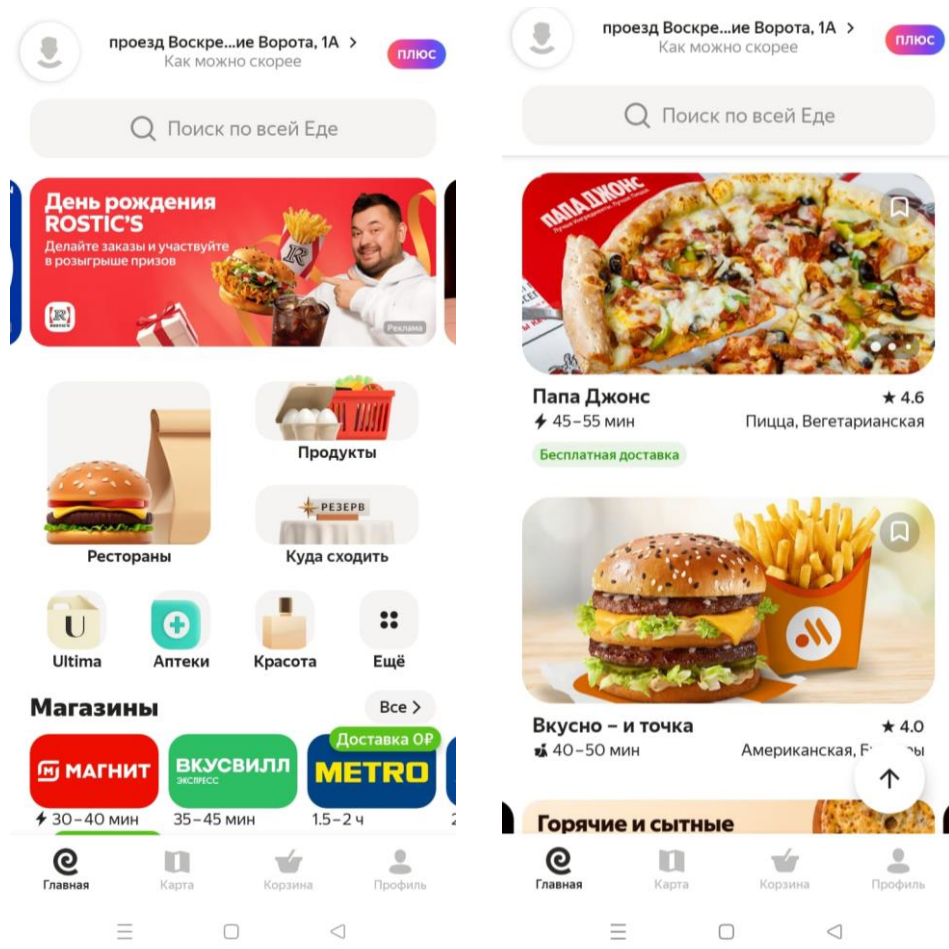
1 Аналитический раздел

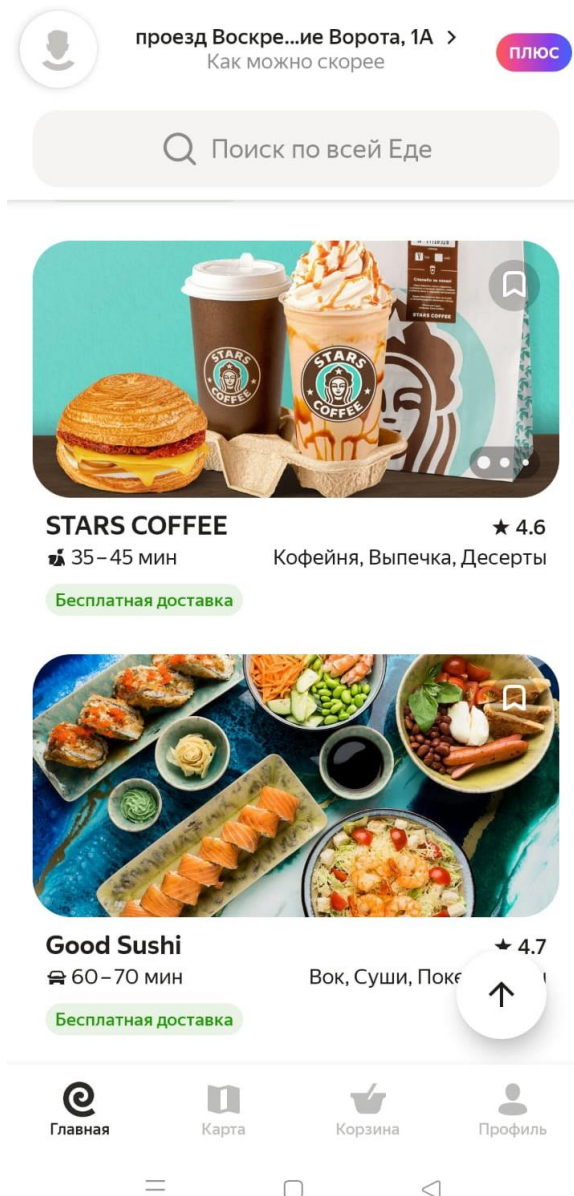
1.1 Анализ существующих решений

На рынке существует множество приложений для доставки еды, таких как **Uber Eats**, **Delivery Club**, **Glovo**, **Яндекс.Еда** и др. Эти сервисы предоставляют пользователям возможность быстро заказывать еду и отслеживать статус доставки. Однако большинство подобных решений являются крупными коммерческими продуктами с ограниченной возможностью кастомизации под конкретные нужды.

В отличие от них, YummiBox предлагает более лёгкое и локализованное решение, ориентированное на небольшие или начинающие бизнесы, которым требуется простая в использовании система управления заказами и продуктами, без сложной и дорогостоящей инфраструктуры.

Яндекс.Еда





- **Функции:**

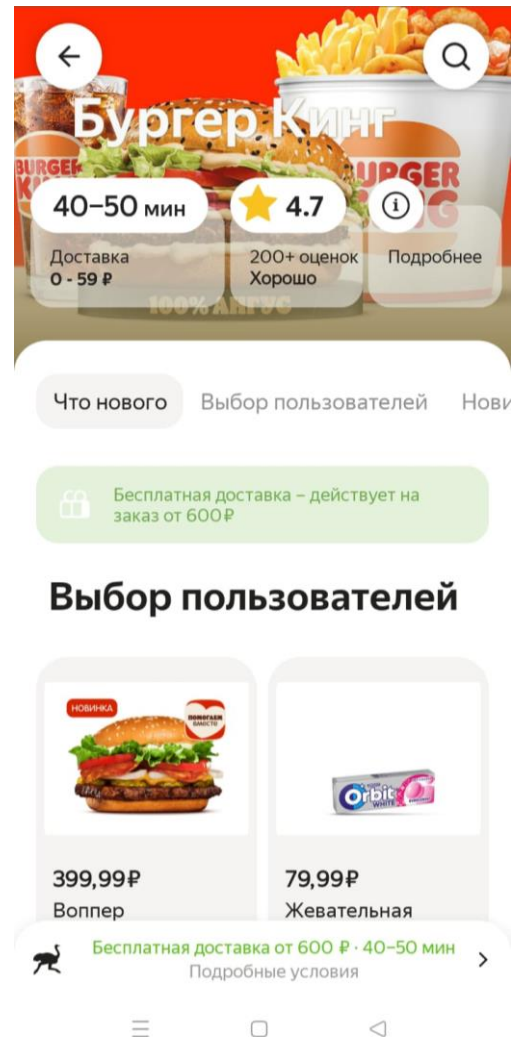
- Поддержка тысяч ресторанов по всей стране;
- Фильтрация по типу кухни, рейтингу, акциям;
- Привязка банковской карты, Apple Pay/Google Pay;
- Отслеживание курьера на карте.

- **Преимущества:**

- Огромная база ресторанов;
- Интеграция с другими сервисами Яндекса;

- Быстрая доставка в крупных городах.
- **Недостатки:**
 - Высокая конкуренция среди заведений;
 - Комиссия для ресторанов — до 30%;
 - Отсутствие прямого контакта клиента с кухней.

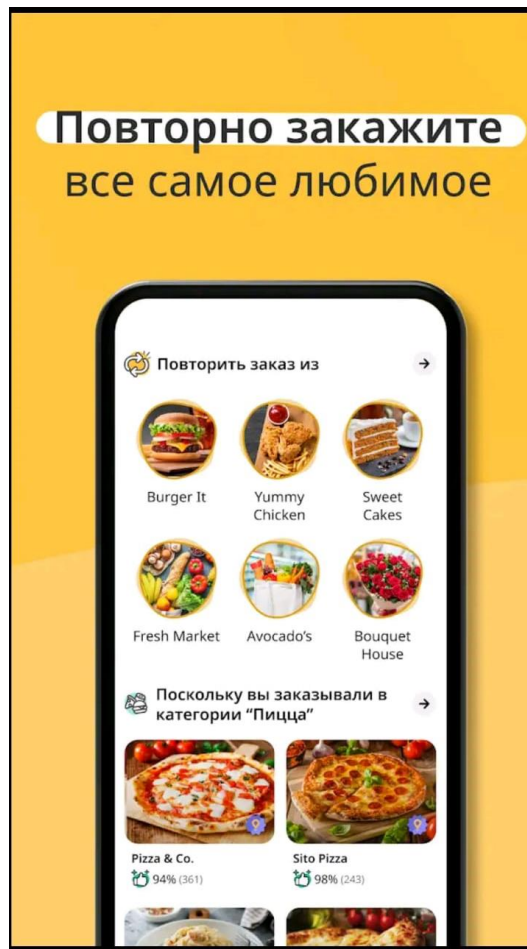
Delivery Club



- **Функции:**
 - Каталог с большим количеством ресторанов;
 - Возможность заказать алкоголь, продукты, подарки;
 - Программа лояльности и бонусы;
 - Поддержка самовывоза.
- **Преимущества:**
 - Хорошее покрытие;

- Удобный интерфейс;
- Множество акций и промокодов.
- **Недостатки:**
 - Перегруженный интерфейс;
 - Нестабильная работа уведомлений;
 - Нет опции "один бренд — всё под контролем", как у YummiBox.

Glovo



- **Функции:**
 - Универсальная доставка (еда, лекарства, техника и т. д.);
 - Возможность заказать «что угодно»;
 - Интерфейс «одного окна» с минималистичным дизайном.
- **Преимущества:**
 - Гибкий подход к типам заказов;
 - Быстрая регистрация и оформление;

- Хорошая навигация по категориям.
- **Недостатки:**
 - Ограниченная география (в России уже не работает официально);
 - Сложность с курьерской логистикой;
 - Не всегда надежные партнеры.

1.2 Выбор архитектуры приложения

Проект был реализован в виде двух отдельных мобильных приложений:

- **Клиентская часть (пользовательское приложение)** — предназначена для оформления заказов, просмотра меню, регистрации, авторизации и взаимодействия с базой данных.
- **Административная панель** — используется для управления категориями товаров, товарами, загрузкой изображений, а также для просмотра заказов и управления ими.

Архитектура основана на принципе "**клиент-сервер**", где мобильные приложения взаимодействуют с удалёнными базами данных и облачным хранилищем через API и SDK Firebase и Supabase.

1.3 Постановка задачи

Требовалось разработать мобильное приложение, включающее:

- Функциональность пользовательской регистрации/авторизации;
- Просмотр и фильтрацию товаров по категориям;
- Добавление товаров в корзину и оформление заказа;
- Административную систему управления (добавление/удаление/редактирование товаров и категорий);
- Загрузку изображений продуктов;
- Отображение списка заказов в реальном времени.

1.4 Обоснование выбора инструментов

Для реализации проекта были выбраны следующие технологии по следующим причинам:

- **Kotlin и Android Studio** — нативная разработка под Android обеспечивает высокую производительность, контроль над интерфейсом и глубокую интеграцию с платформой;
- **Firebase Realtime Database** — обеспечивает синхронизацию данных в реальном времени, что удобно для отображения и обновления заказов;
- **Firebase Authentication** — упрощает реализацию безопасной регистрации и входа в приложение;
- **Supabase Storage** — легко интегрируется и позволяет безопасно хранить изображения товаров;
- **Glide** — оптимизирует загрузку и отображение изображений, повышая производительность приложения.

2 Проектирование системы

2.1 Структура пользовательского приложения

Клиентское приложение YummiBox разрабатывалось с упором на простоту и удобство использования. Оно включает следующие основные разделы:

- **Экран входа и регистрации** – с использованием Firebase Authentication;
- **Главная страница** – отображает категории и список доступных продуктов;
- **Карточка продукта** – содержит изображение, описание, цену, кнопку «Добавить в корзину»;
- **Корзина** – позволяет просматривать, редактировать и подтверждать заказы;
- **Профиль пользователя** – отображает информацию об учётной записи и историю заказов.

Навигация реализована с использованием стандартных Android-компонентов (BottomNavigationView, Fragments, ViewModel), что упрощает переход между экранами.

2.2 Структура административной панели

Административная панель также реализована в виде Android-приложения.

Основные функции:

- **Авторизация администратора;**
- **Добавление, редактирование и удаление категорий и товаров;**
- **Загрузка изображений товаров в Supabase;**
- **Просмотр активных заказов в реальном времени (через Firebase Realtime Database).**

Разделы приложения реализованы по принципу Activity + Fragment, где каждая сущность (продукт, категория, заказ) имеет собственный интерфейс управления.

2.3 Модель базы данных Firebase

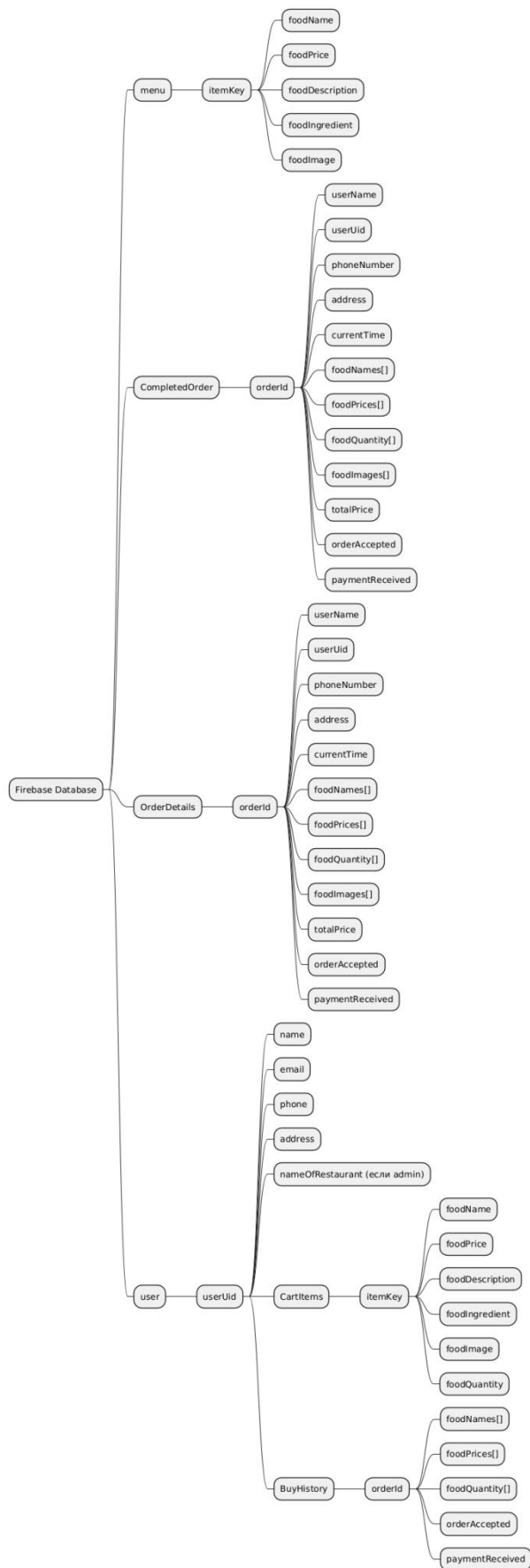


рис.1 Модель базы данных Firebase

Диаграмма визуализирует иерархическую структуру хранения данных проекта YummiBox в Firebase Realtime Database. Выбранная модель организована по ветвям, каждая из которых отвечает за конкретный аспект приложения.

➤ **menu**

Хранит список блюд, доступных пользователям. Каждый элемент (**itemKey**) содержит:

- **foodName** — название блюда;
- **foodPrice** — цена;
- **foodDescription** — краткое описание;
- **foodIngredient** — список ингредиентов;
- **foodImage** — ссылка на изображение блюда (загружается через Supabase).

Эта ветка используется для отображения меню в клиентском приложении.

➤ **CompletedOrder**

Содержит завершённые или оформленные заказы. Структура включает:

- **orderId** — уникальный идентификатор;
- **userName, userId, phoneNumber, address** — данные пользователя;
- **foodNames[], foodPrices[], foodQuantity[], foodImages[]** — списки заказанных блюд;
- **totalPrice** — общая сумма;
- **orderAccepted, paymentReceived** — статусы обработки.

Эти данные видны в админ-панели для подтверждения и контроля заказов.

➤ **OrderDetails**

Аналогично **CompletedOrder**, но может представлять текущие или активные заказы, находящиеся в обработке. Используется для отображения заказов в реальном времени до их завершения.

➤ **user**

Ветка, хранящая информацию о зарегистрированных пользователях. Каждому **userId** соответствует:

- Персональные данные: **name, email, phone, address**;
- **nameOfRestaurant** — только для администраторов;
- **CartItems** — товары, добавленные в корзину, с полями блюда (аналогично **menu**);
- **BuyHistory** — история покупок, где хранятся сведения о предыдущих заказах (имена, цены, количество, статус).

Это позволяет персонализировать интерфейс, сохранять сессии и быстро получать доступ к действиям пользователя.

Особенности реализации:

- Структура соответствует носимому характеру **Firebase**, где чтение/запись данных осуществляется по **URL**-ветке;
- Использование массивов (**foodNames[]** и т. д.) позволяет удобно сериализовать списки;
- Разделение на **OrderDetails** и **CompletedOrder** обеспечивает этапность обработки заказов;
- **Firebase Authentication** даёт доступ к уникальному **userId**, который является ключом.

2.4 Диаграмма вариантов использования (Use Case)

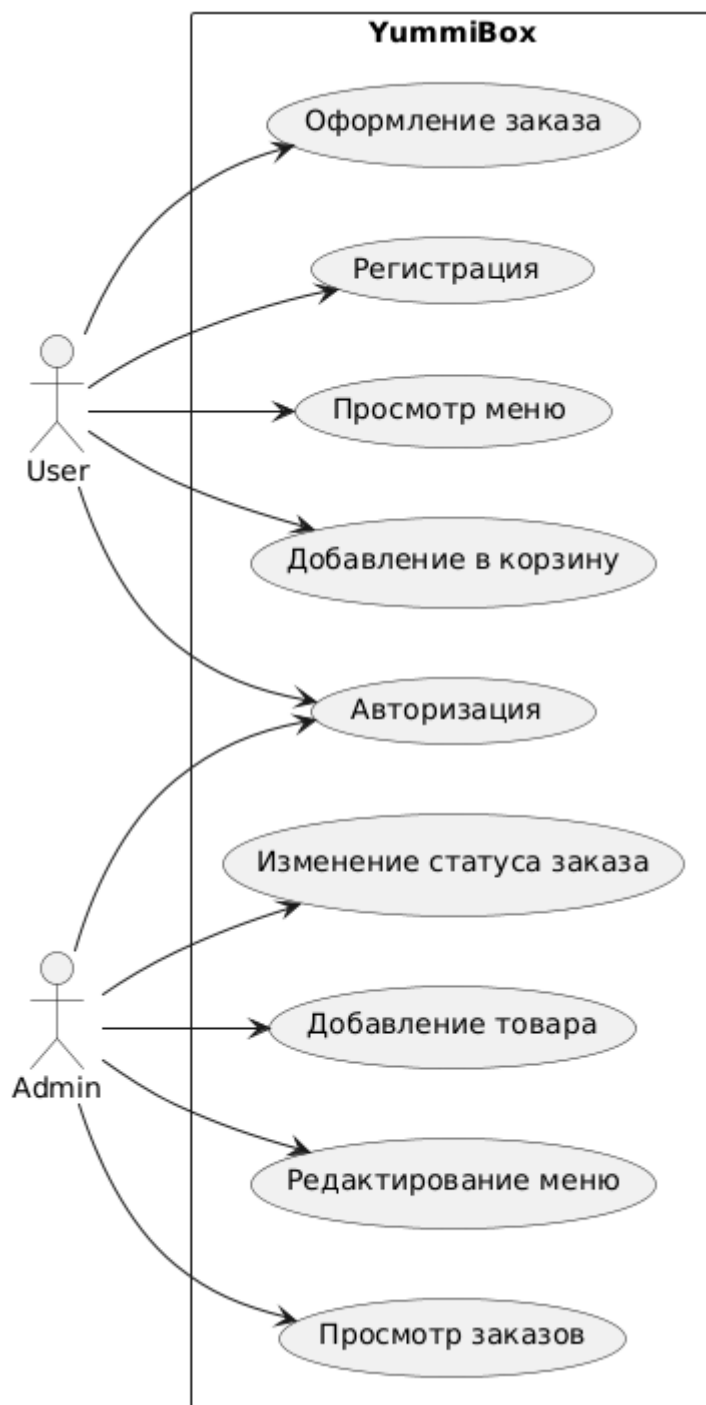


Диаграмма вариантов использования позволяет отразить функциональные возможности системы YummiBox, доступные для двух типов пользователей — пользователя и администратора. Она показывает, кто какие действия может выполнять.

Основные участники (Actors):

- **User (Пользователь)** — клиент, оформляющий заказы;
- **Admin (Администратор)** — сотрудник или владелец ресторана, управляющий меню и заказами.

Возможности пользователя:

- **Регистрация** — создание нового аккаунта;
- **Авторизация** — вход в систему с учётными данными;
- **Просмотр меню** — отображение списка блюд, полученных из Firebase;
- **Добавление в корзину** — сохранение выбранных блюд;
- **Оформление заказа** — отправка информации о заказе в базу данных.

Возможности администратора:

- **Авторизация** — доступ к административной панели;
- **Добавление товара** — ввод нового блюда в меню;
- **Редактирование меню** — изменение существующих позиций;
- **Просмотр заказов** — работа с поступившими заказами;
- **Изменение статуса заказа** — перевод заказов в состояние "принят", "оплачен", "завершён".

2.5 Диаграмма потоков данных (DFD)

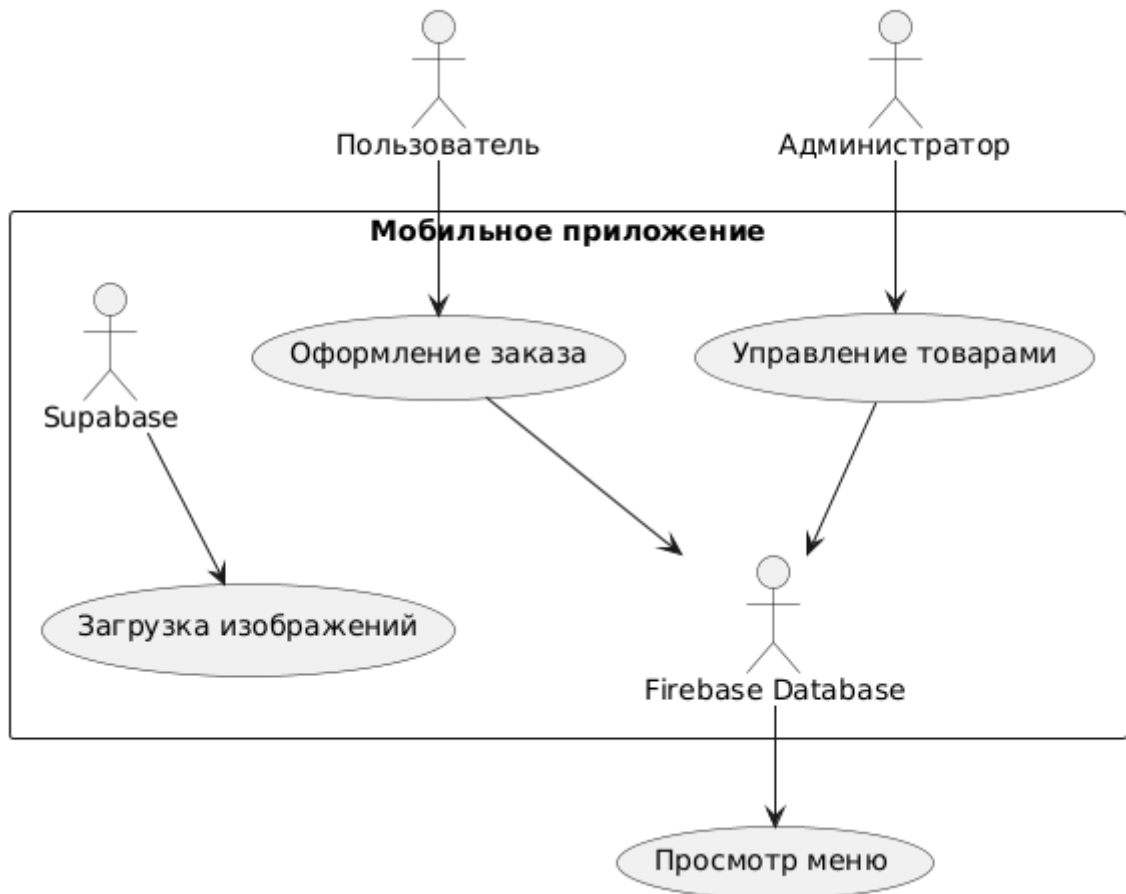


Диаграмма потоков данных (Data Flow Diagram) отображает движение информации между основными компонентами системы. Она помогает понять, какие данные куда передаются, откуда приходят и какие действия их вызывают.

Основные элементы:

- **Акторы:**
 - *Пользователь* — инициирует заказы;
 - *Администратор* — управляет содержимым меню и заказами.
- **Процессы:**
 - *Оформление заказа* — создаётся запись в Firebase;
 - *Управление товарами* — администратор добавляет/редактирует блюда;
 - *Просмотр меню* — приложение загружает список из Firebase.
- **Хранилища и интеграции:**

- *Firebase Database* — основная база хранения меню, заказов, пользователей;
- *Supabase* — отдельное облачное хранилище изображений товаров.

Потоки:

1. Пользователь отправляет запросы: заказать блюдо, просмотреть меню.
2. Приложение взаимодействует с Firebase: читает меню, записывает заказ.
3. Администратор управляет меню — это обновляет данные в Firebase.
4. Supabase используется для загрузки и отображения изображений.

2.6 Диаграмма классов (Class Diagram)

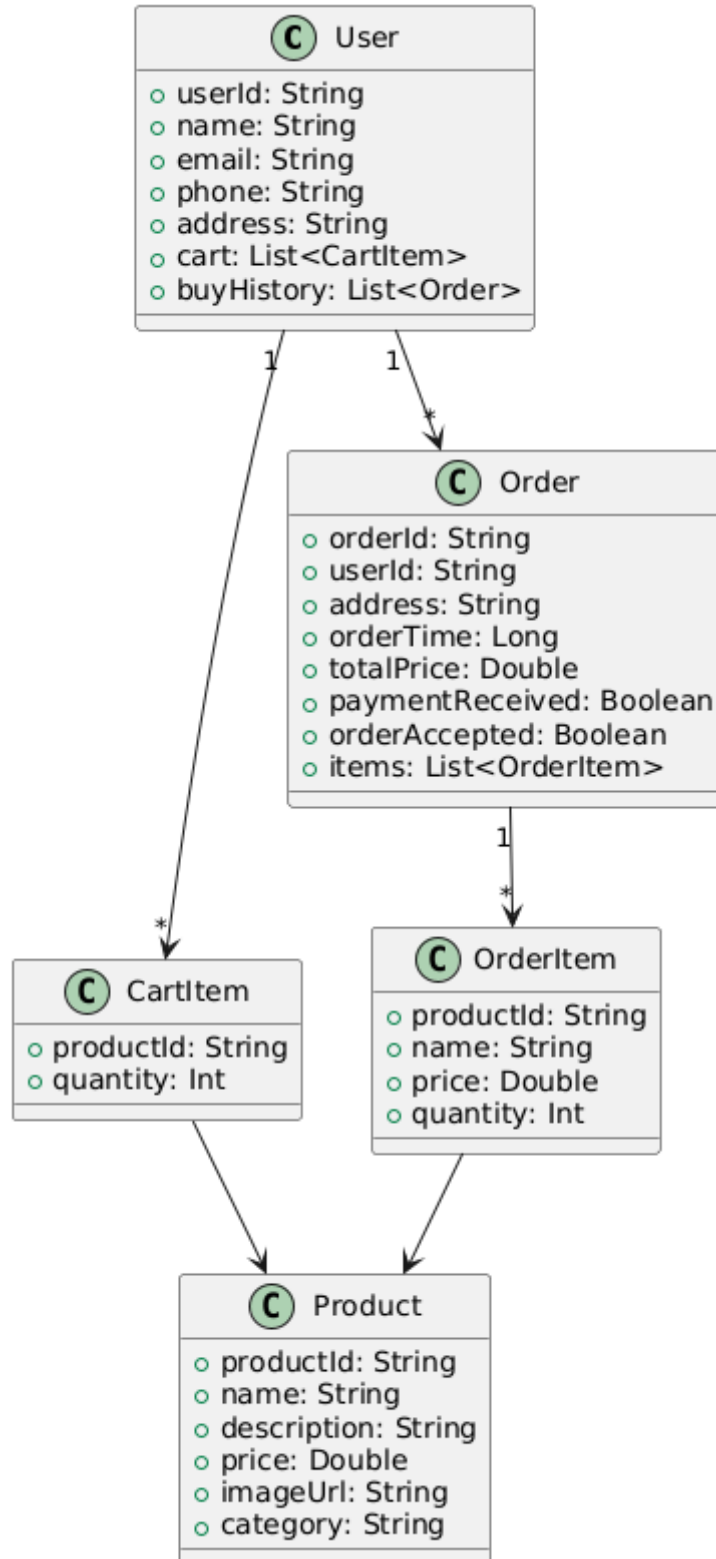


Диаграмма классов отражает логическую структуру основных объектов системы YummiBox и взаимосвязи между ними.

В центре модели — сущность **User**, которая может иметь множество заказов (**Order**) и элементов корзины (**CartItem**). Класс **Order** содержит

информацию о заказе: уникальный идентификатор, дату, статус, итоговую стоимость, а также список заказанных товаров (OrderItem). Последние, в свою очередь, ссылаются на Product, содержащий описание блюда, цену и изображение.

Такое проектирование обеспечивает хорошую модульность, повторное использование объектов и удобство работы с Firebase-структурой.

2.7 Диаграмма последовательности (Sequence Diagram)

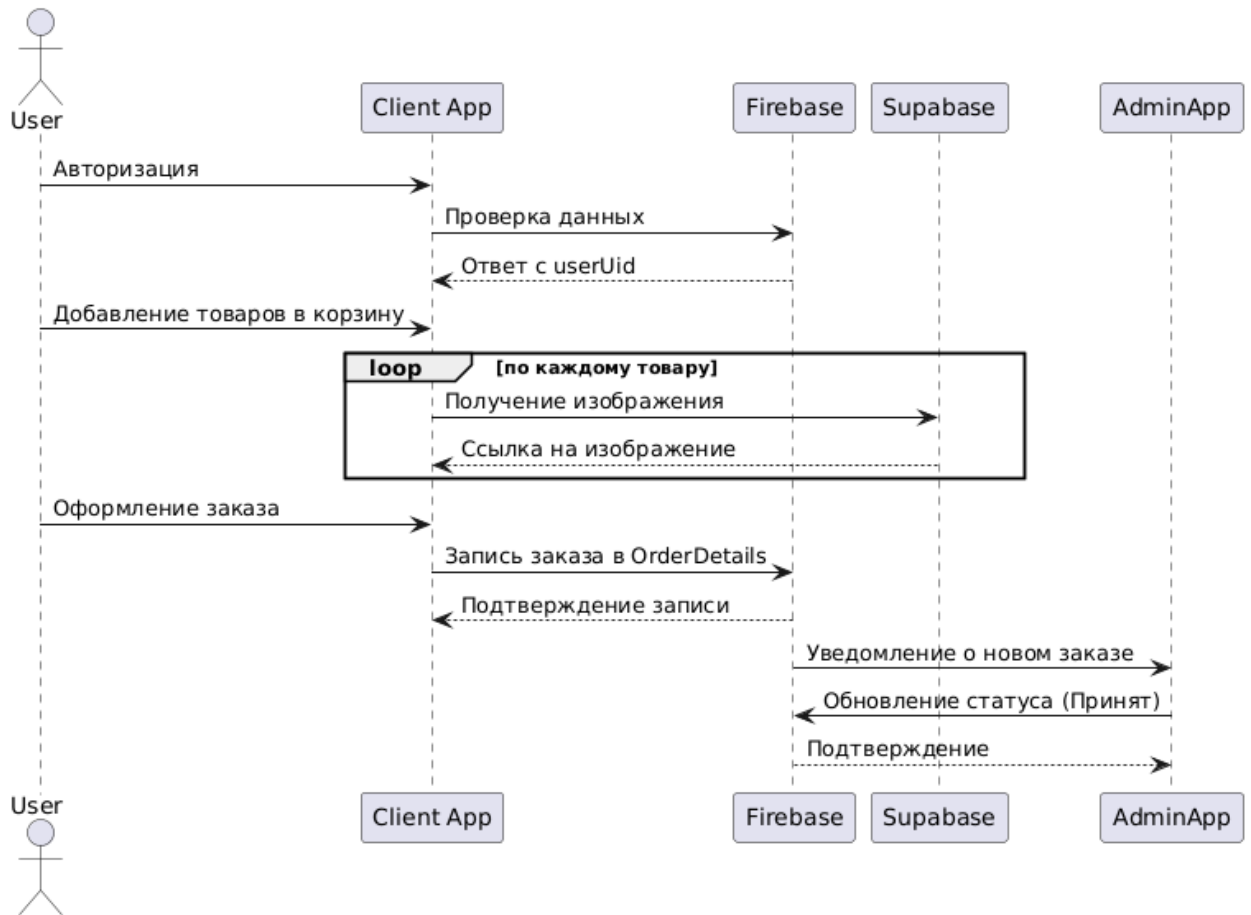


Диаграмма последовательности описывает жизненный цикл оформления заказа — от входа пользователя в систему до обновления статуса в админке.

Сценарий включает взаимодействие между актёрами (пользователь, клиентское приложение, Firebase, Supabase и административная панель).

Приложение сначала получает данные пользователя, затем — изображения блюд с Supabase. После подтверждения заказа он отправляется в Firebase, где система сохраняет данные в ветке OrderDetails. Далее админ-панель получает уведомление и может обновить статус — это отражает работу в режиме реального времени.

2.8 Диаграмма состояний (State Diagram)

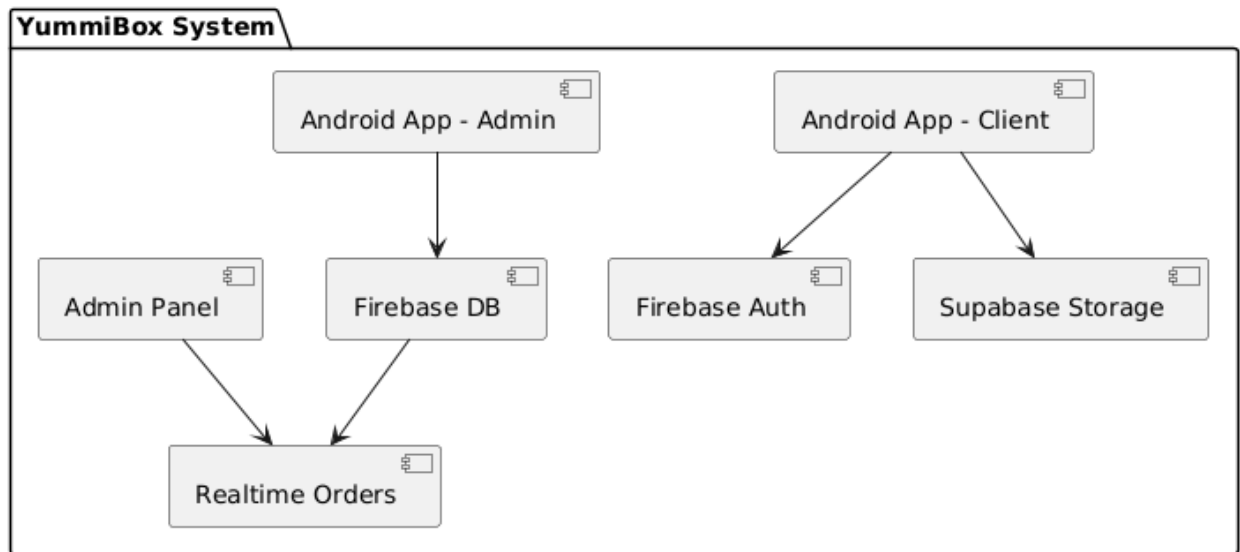


Диаграмма состояний описывает, какие этапы проходит заказ после его создания. Изначально заказ имеет статус "Новый", далее переходит в "Принят" после подтверждения администратором. Если клиент производит оплату — заказ переходит в состояние "Оплачен", и, наконец, "Завершён" — после доставки.

Возможен альтернативный путь — отмена заказа, что переводит его в состояние "Отменён".

Такая модель проста и гибка, её можно расширять (например, добавлением состояния "Готовится" или "В доставке").

2.9 Диаграмма компонентов системы



2.10 Диаграммы потоков данных

Потоки данных проектируются следующим образом:

- **Пользователь** → отправляет запрос (регистрация, заказ, вход);
- **Приложение** → обращается к Firebase (аутентификация, чтение/запись данных);
- **Firebase/Supabase** → возвращает данные или статус запроса;
- **Приложение** → отображает результат пользователю или администратору.

В административной панели данные также обновляются в реальном времени, что позволяет видеть заказы по мере их поступления.

2.11 Прототипы интерфейсов

На этапе предварительного проектирования интерфейсы мобильного приложения и административной панели были разработаны в инструменте [Figma](#), что позволило:

- визуализировать пользовательские потоки;
- провести предварительное тестирование интерфейса;
- согласовать дизайн до начала программной реализации.

Были созданы отдельные макеты для:

- главного экрана приложения;
- карточки товара;
- экрана авторизации и регистрации;
- корзины;
- административных форм (добавление товара, список заказов и т.д.).

Ключевые принципы проектирования:

- ☐ **Адаптивность** — макеты оптимизированы под различные разрешения экранов смартфонов;
- ☐ **Фокус на UX** — минималистичный и интуитивный интерфейс без перегрузки элементов;
- ☐ **Разделение ролей** — пользователь видит только функции заказа, администратор — управление данными.

Figma позволила легко экспортировать готовые ресурсы (иконки, изображения, цвета) в Android Studio и значительно ускорить верстку интерфейсов.

3 Реализация проекта

3.1 Реализация клиентского мобильного приложения

Клиентская часть YummiBox была разработана с использованием **Android Studio** на языке **Kotlin**. Главный акцент сделан на простоте интерфейса, высокой отзывчивости и стабильной работе с сетью.

Основные экраны:

- **Авторизация/регистрация** — реализована через Firebase Authentication;
- **Главная страница** — отображает список категорий и продуктов (данные из Realtime Database);
- **Карточка товара** — показывает изображение, описание, цену, ингредиенты, кнопку добавления в корзину;
- **Корзина** — отображает выбранные товары, их количество и общую сумму;
- **Оформление заказа** — данные отправляются в ветку OrderDetails базы Firebase.

Для загрузки и отображения изображений используется библиотека **Glide**, что обеспечивает кэширование и быструю подгрузку медиафайлов из **Supabase Storage**.

3.2 Реализация административной панели

Админская панель — это отдельное Android-приложение, предназначенное только для уполномоченных пользователей.

Ключевой функционал:

- Авторизация админа (по email/пароллю);
- Добавление и редактирование блюд (загрузка изображения в Supabase, запись описания, цены, ингредиентов и т.д.);
- Просмотр заказов в реальном времени (из ветки CompletedOrder);
- Изменение статуса заказа (подтверждение, оплата и т.д.);
- Удаление или изменение товаров в меню.

Формы реализованы с валидацией данных. При добавлении изображений сначала происходит загрузка в Supabase, после чего ссылка записывается в базу Firebase.

3.3 Интеграция с Firebase и Supabase

Используемые модули:

- **Firebase Authentication** — регистрация, вход, проверка сессии;
- **Firebase Realtime Database** — хранение меню, заказов, корзины и истории покупок;
- **Supabase Storage** — используется как внешний файловый CDN-хостинг для изображений;
- **Firebase SDK** — встроен через Gradle и обрабатывает чтение/запись с помощью ValueEventListener.

Доступ к базе данных и хранилищу реализован с учётом прав доступа, что исключает несанкционированное изменение данных.

Для интеграции облачных сервисов в приложение YummiBox были использованы консольные панели Firebase и Supabase:

- Firebase Authentication:
<https://console.firebase.google.com/project/yummibox/authentication/users>
- Firebase Realtime Database:
<https://console.firebase.google.com/project/yummibox/database/>
- Supabase Storage (хранилище изображений):
<https://supabase.com/dashboard/project/wjejxjzodprauxdjtiys/storage/buckets/images>

Процесс подключения Firebase:

- Проект был создан в консоли Firebase;
- Подключение SDK выполнялось через google-services.json в корне Android-проекта;
- Аутентификация, БД и разрешения были настроены в консоли;

- Была включена email-аутентификация и настроены базовые правила доступа к данным.

Работа с Supabase:

- Используется только модуль Storage;
- Загруженные изображения помещаются в публичный бакет;
- После загрузки клиент получает публичный URL изображения, который сохраняется в Firebase;
- Для загрузки использован HttpURLConnection и REST API Supabase.

Архитектурное обоснование:

Firebase служит базой бизнес-логики: авторизация, заказы, корзина.

Supabase — файловое хранилище. Разделение позволяет уменьшить нагрузку на Firebase и использовать более гибкую систему управления файлами.

Такой подход обеспечил:

- масштабируемость и лёгкость обслуживания;
- независимость логики и хранения файлов;
- возможность подключения к другим облачным модулям в будущем.

3.4 Авторизация и регистрация пользователей

Пользователи проходят регистрацию по email и паролю. После успешной аутентификации Firebase выдаёт уникальный `userId`, на основе которого создаются соответствующие записи в базе:

- данные профиля;
- корзина;
- история покупок.

Для каждого пользователя создаётся своя структура внутри `user/userId/`.

3.5 Загрузка и отображение изображений (Glide + Supabase)

При добавлении нового блюда:

1. Администратор выбирает изображение с устройства;

2. Оно загружается в Supabase через публичную ссылку;
3. Ссылка сохраняется в Firebase Realtime Database;
4. Клиентское приложение подгружает изображения через библиотеку **Glide**, что обеспечивает оптимальную скорость и масштабирование под устройство пользователя.

4 Анализ ключевых фрагментов кода

1 Загрузка изображения в Supabase и сохранение в Firebase

Фрагмент кода (Kotlin, Admin-YummiBox)

```
private fun uploadImageToSupabase(uri: Uri, onSuccess: (imageUrl:
String) -> Unit) {
    val inputStream = contentResolver.openInputStream(uri)
    val bitmap = BitmapFactory.decodeStream(inputStream)
    inputStream?.close()

    val outputStream = java.io.ByteArrayOutputStream()
    bitmap.compress(Bitmap.CompressFormat.JPEG, 50, outputStream)
    val compressedBytes = outputStream.toByteArray()

    val requestFile =
compressedBytes.toRequestBody("image/jpeg".toMediaTypeOrNull())
    val fileName = "compressed_${System.currentTimeMillis()}.jpg"
    val body = MultipartBody.Part.createFormData("file", fileName,
requestFile)

    val call = SupabaseClient.service.uploadImage(
        SupabaseClient.getApiKey(),
        SupabaseClient.getAuthorization(),
        body,
        "images",
        fileName
    )
}
```

```

        call.enqueue(object : Callback<ResponseBody> {
            override fun onResponse(call: Call<ResponseBody>, response:
Response<ResponseBody>) {
                if (response.isSuccessful) {
//                    val imageUrl =
"https://supabase.com/dashboard/project/wjejxjzodprauxdjtiys/storage/bucke
ts/images/$fileName"
                    val imageUrl =
"https://wjejxjzodprauxdjtiys.supabase.co/storage/v1/object/public/images/$f
ileName"
//                    Glide.with(this@AddItemActivity)
//                        .load(imageUrl)
//                        .into(binding.selectedImage)
                        onSuccess(imageUrl)
                } else {
                    Log.e("Supabase", "Upload failed: code=${response.code()}
body=${response.errorBody()?.string()}")
                    Toast.makeText(this@AddItemActivity, "Failed uploading
image: ${response.code()}", Toast.LENGTH_SHORT).show()
                }
            }
        })

        override fun onFailure(call: Call<ResponseBody>, t: Throwable) {
            Toast.makeText(this@AddItemActivity, "Error: ${t.message}",
Toast.LENGTH_SHORT).show()
        }
    })
}

```

Пояснение

Данный метод реализует **отправку изображения на облачное хранилище Supabase**. Он используется в административной части приложения YummiBox при добавлении нового блюда.

Основные этапы:

1. Получение и сжатие изображения:

- URI изображения передаётся в метод;
- С помощью `BitmapFactory.decodeStream` изображение декодируется;
- Изображение сжимается до 50% качества с использованием JPEG.

2. Подготовка запроса:

- Формируется `MultipartBody.Part`, необходимый для отправки изображения как части формы;
- Создаётся уникальное имя файла на основе метки времени.

3. Вызов Supabase API:

- Метод `uploadImage` вызывается через интерфейс `Retrofit`;
- Используются заголовки авторизации (API key и Bearer token);
- Изображение отправляется в публичный бакет `images`.

4. Обработка ответа:

- При успешной загрузке создаётся URL вида:
`https://[project-id].supabase.co/storage/v1/object/public/images/<имя_файла>`
- URL передаётся в `onSuccess()` и впоследствии сохраняется в

Firebase вместе с данными блюда.

Преимущества подхода:

- Сжатие перед загрузкой экономит трафик;
- Использование `enqueue()` позволяет выполнять операцию асинхронно, не блокируя UI;
- Публичные ссылки Supabase позволяют отображать изображения сразу после загрузки.

2 Авторизация администратора и сохранение данных в Firebase

Ключевые фрагменты кода (Kotlin)

```
auth.createUserWithEmailAndPassword(email, password)

    .addOnCompleteListener { task ->

        if (task.isSuccessful) {

            val userId = FirebaseAuth.getInstance().currentUser!!.uid

            val user = UserModel(userName, nameOfRestaurant, email,
password)

            database.child("user").child(userId).setValue(user)

        }

    }
```

Авторизация с fallback на создание пользователя:

```
auth.signInWithEmailAndPassword(email, password)

    .addOnCompleteListener { task ->

        if (task.isSuccessful) {

            updateUi(auth.currentUser)

        }

    }
```

```

    } else {

        // Попытка создать аккаунт, если не существует

        auth.createUserWithEmailAndPassword(email, password)

            .addOnCompleteListener { createTask ->

                if (createTask.isSuccessful) {

                    saveUserData()

                    updateUi(auth.currentUser)

                }

            }

        }

    }
}

```

Авторизация через Google:

```

val credential = GoogleAuthProvider.getCredential(account.idToken, null)

auth.signInWithCredential(credential)

    .addOnCompleteListener { authTask ->

        if (authTask.isSuccessful) {

            updateUi(authTask.result?.user)

        }

    }
}

```

В административной части приложения реализованы **три способа авторизации**:

1. Регистрация с email/паролем

Пользователь (администратор ресторана) вводит свои данные, включая

название ресторана, email и пароль. После успешного создания аккаунта вызывается метод `saveUserData()`, который сохраняет объект `UserModel` в `Firebase Realtime Database` по пути `user/userId/`.

2. Авторизация

При входе используется `signInWithEmailAndPassword(...)`. Если аккаунта ещё не существует, приложение автоматически пытается его создать — это повышает удобство использования. После успешного входа вызывается `updateUi(user)`, перенаправляющая на основной экран (`MainActivity`).

3. Google Sign-In

Дополнительно реализована авторизация через Google. После успешной аутентификации через Google полученный `idToken` используется для создания `FirebaseAuthCredential`, после чего пользователь считается вошедшим. Это реализовано через `ActivityResultLauncher` и `GoogleSignInClient`.

Хранение в Firebase

После регистрации или авторизации данные администратора сохраняются в ветке `user/userUid`, где `UserModel` содержит:

- имя пользователя;
- email;
- пароль (в виде строки — рекомендуется заменить на безопасный токен);
- название ресторана.

3 Работа с корзиной и формирование заказа клиентом

Назначение

В клиентской части приложения `YummiBox` реализована логика,

позволяющая пользователю просматривать содержимое своей корзины, редактировать его, а затем передавать выбранные блюда на следующий этап оформления заказа. Эти процессы реализованы в методах `reteriveCartItems()` и `getOrderItemsDetail()`.

Ключевые фрагменты кода (Kotlin)

```
private fun reteriveCartItems() {  
    // database refernce to the Firebase  
    database = FirebaseDatabase.getInstance()  
    userId = auth.currentUser?.uid?:""  
    val foodReference: DatabaseReference =  
database.reference.child("user").child(userId).child("CartItems")  
  
    // list to store cart items  
    foodNames = mutableListOf()  
    foodPrices = mutableListOf()  
    foodImagesUri = mutableListOf()  
    foodDescriptions = mutableListOf()  
    foodIngredients = mutableListOf()  
    foodQuantity = mutableListOf()  
  
    //fetch data from the database  
    foodReference.addListenerForSingleValueEvent(object  
: ValueEventListener {  
        override fun onDataChange(snapshot: DataSnapshot) {  
            for (foodSnapshot in snapshot.children) {  
                // get the cartItems object from the child node  
                val cartItems = foodSnapshot.getValue(CartItems::class.java)  
                //add cart items details to the list  
                cartItems?.foodName?.let { foodNames.add(it) }  
            }  
        }  
    })  
}
```

```

        cartItems?.foodPrice?.let { foodPrices.add(it) }
        cartItems?.foodImage?.let { foodImagesUri.add(it) }
        cartItems?.foodDescription?.let { foodDescriptions.add(it) }
        cartItems?.foodIngredient?.let { foodIngredients.add(it) }
        cartItems?.foodQuantity?.let { foodQuantity.add(it) }
    }
    setAdapter()
}

private fun setAdapter(){
    cartAdapter = CartAdapter(requireContext(), foodNames, foodPrices,
    foodImagesUri, foodDescriptions, foodIngredients, foodQuantity)
    binding.cartRecyclerView.layoutManager =
    LinearLayoutManager(requireContext(), LinearLayoutManager.VERTICAL,
    false)
    binding.cartRecyclerView.adapter = cartAdapter
}

    override fun onCancelled(error: DatabaseError) {
        Toast.makeText(context, "Data not fetch",
        Toast.LENGTH_SHORT).show()
    }

})
}

```

Этот метод отвечает за загрузку данных из Firebase Realtime Database из ветки:

user/{userId}/CartItems

Основные действия:

- Получение текущего `userId` через `FirebaseAuth`.
- Считывание всех записей корзины пользователя.
- Заполнение списков: `foodNames`, `foodPrices`, `foodImagesUri`, `foodDescriptions`, `foodIngredients`, `foodQuantity`.
- Инициализация и установка `CartAdapter` с этими данными для отображения в `RecyclerView`.

Используется однократное чтение (`addListenerForSingleValueEvent`), что снижает нагрузку и подходит для локальной обработки.

Результат:

Пользователь видит список всех добавленных блюд с изображением, ценой, описанием и ингредиентами.

Метод `getOrderItemsDetail()`

```
private fun getOrderItemsDetail() {
    val orderIdReference: DatabaseReference =
        database.reference.child("user").child(userId).child("CartItems")

    val foodName = mutableListOf<String>()
    val foodPrice = mutableListOf<String>()
    val foodImage = mutableListOf<String>()
    val foodDescription = mutableListOf<String>()
    val foodIngredient = mutableListOf<String>()
    // get item Quantities
    val foodQuantities = cartAdapter.getUpdatedItemsQuantities()

    orderIdReference.addListenerForSingleValueEvent(object
        : ValueEventListener {
        override fun onDataChange(snapshot: DataSnapshot) {
```

```

        for (foodSnapshot in snapshot.children){
            // get the cartItems to respective list
            val orderItems = foodSnapshot.getValue(CartItems::class.java)
            // add items details in to list
            orderItems?.foodName?.let { foodName.add(it) }
            orderItems?.foodPrice?.let { foodPrice.add(it) }
            orderItems?.foodImage?.let { foodImage.add(it) }
            orderItems?.foodDescription?.let { foodDescription.add(it) }
            orderItems?.foodIngredient?.let { foodIngredient.add(it) }
        }
        orderNow(foodName, foodPrice, foodImage, foodDescription,
        foodIngredient, foodQuantities)
    }

    override fun onCancelled(error: DatabaseError) {
        Toast.makeText(requireContext(), "Order making Failed, please try
        again", Toast.LENGTH_SHORT).show()
    }
}

```

Этот метод вызывается при нажатии кнопки "Proceed" для перехода к оформлению заказа. Его задача — **повторно прочитать актуальные данные корзины** и дополнительно получить **фактическое количество каждого блюда**, введённое пользователем (через `cartAdapter.getUpdatedItemsQuantites()`).

Последовательность:

- Повторное подключение к ветке CartItems в Firebase.

- Чтение и сбор данных из каждого дочернего узла.
- Передача списков (названия, цены, изображения, количества и др.) в метод `orderNow(...)`, который инициирует переход к активности оплаты — `PayOutActivity`.

Итог

Описанные методы реализуют:

- Получение и отображение содержимого корзины.
- Сбор заказанных данных для оформления.
- Устойчивую работу с Firebase.
- Чёткое разделение обязанностей между чтением, отображением и передачей данных.

Эта архитектура облегчает дальнейшую поддержку, масштабирование и внесение новых функций (например, скидки, акции, удаление отдельных блюд и т.п.).

5 Тестирование и отладка

5.1 Методика тестирования

Для обеспечения стабильности и корректной работы приложения использовалось **ручное тестирование** (manual testing) на этапах разработки, а также элементы **интеграционного тестирования**.

Тестирование проводилось по следующим направлениям:

- регистрация и вход в систему;
- добавление и удаление товаров;
- оформление заказов;
- сохранение и отображение изображений;
- обновление статуса заказов в реальном времени;
- корректная работа при отсутствии подключения к сети.

Использовались как **реальные Android-устройства**, так и **эмуляторы** (Pixel, Samsung).

5.2 Тест-кейсы и результаты

Примеры тест-кейсов:

№	Название теста	Входные данные	Ожидаемый результат	Статус
1	Регистрация нового пользователя	Email + пароль	Переход на главный экран	☐ Пройден
2	Добавление блюда администратором	Название, цена, фото	Отображение нового блюда в клиентском UI	☐ Пройден
3	Оформление заказа	Корзина заполнена	Создание записи в "OrderDetails"	☐ Пройден
4	Отображение изображений	URL из Supabase	Картинка загружена через Glide	☐ Пройден
5	Обработка потери соединения	Отключение Wi-Fi	Сообщение об ошибке, повтор при восстановлении	☐ Пройден

Ошибки, возникавшие в процессе тестирования:

- Проблемы с отображением нестандартных символов в названиях блюд — решено использованием кодировки UTF-8.
- Задержка при загрузке большого количества изображений — оптимизирована через кэш Glide.
- Повторная отправка заказа при нестабильном соединении — решено блокировкой кнопки после первого нажатия.

5.3 Обнаруженные ошибки и способы их устранения

- **Ошибка:** при отмене добавления изображения ссылка всё равно сохранялась в базе.
Решение: добавлена проверка на успешность загрузки перед записью URL.
- **Ошибка:** заказ не обновлялся в админке без перезапуска.
Решение: реализовано прослушивание изменений в ветке CompletedOrder с помощью ValueEventListener.
- **Ошибка:** неверно считалась общая сумма заказа.
Решение: пересчитан алгоритм суммирования с учётом количества каждого блюда.

6 Экономическая часть

6.1 Расчёт затрат на разработку

Так как проект разрабатывался самостоятельно, в рамках учебного процесса, основные затраты связаны с временными и условными ресурсами, а не с фактическими финансовыми расходами. Тем не менее, при моделировании реальной стоимости разработки проекта были учтены следующие компоненты:

Статья затрат	Обоснование	Условная сумма (руб)
Разработка клиентского приложения	120 ч × 500 руб/ч	60 000
Разработка админ-панели	80 ч × 500 руб/ч	40 000
Дизайн интерфейсов (Figma)	20 ч × 500 руб/ч	10 000
Тестирование и отладка	40 ч × 500 руб/ч	20 000
Итого		130 000

Примечание: ставки и часы выбраны условно и могут варьироваться в зависимости от региона, уровня специалистов и задач.

6.2 Оценка экономической эффективности

Если рассматривать возможность коммерциализации проекта (например, как сервис доставки еды для малого бизнеса), то приложение может:

- сократить расходы бизнеса на разработку ИТ-инфраструктуры;
- предоставить клиентам современный инструмент для онлайн-заказов;
- снизить нагрузку на персонал за счёт автоматизации процессов оформления и обработки заказов.

Окупаемость проекта возможна уже при запуске на небольшом локальном рынке, при условии активного использования системы ресторанами или кафе.

Заключение

В ходе выполнения дипломного проекта была разработана полноценная система для мобильного взаимодействия с сервисом доставки еды, состоящая из двух взаимосвязанных компонентов:

- клиентского приложения для пользователей,
- административной панели для управления содержимым и заказами.

Проект охватывает полный цикл: от проектирования интерфейсов в **Figma** до реализации функционала с использованием **Android (Kotlin)**, **Firebase Realtime Database**, **Firebase Authentication**, **Supabase Storage** и **Glide**. Обеспечена регистрация, авторизация, создание и просмотр заказов, управление товарами и визуальное сопровождение с помощью изображений.

Была проведена всесторонняя отладка и ручное тестирование всех функций, что подтвердило надёжность и стабильность системы. Также была выполнена базовая экономическая оценка, которая показывает потенциал коммерческого применения проекта в малом бизнесе.

В результате:

- реализован пользовательский и административный интерфейс;
- обеспечена синхронизация данных в реальном времени;
- разработка проведена с нуля без использования платных фреймворков;
- получен ценный опыт работы с современными инструментами мобильной и облачной разработки.

1 Перспективы развития проекта

Проект может быть расширен следующими направлениями:

- внедрение оплаты через платёжные сервисы (например, Stripe или PayPal);
- реализация push-уведомлений о статусе заказов;
- создание web-версии административной панели;
- поддержка мультиязычного интерфейса;

- автоматическое формирование отчётов о заказах.

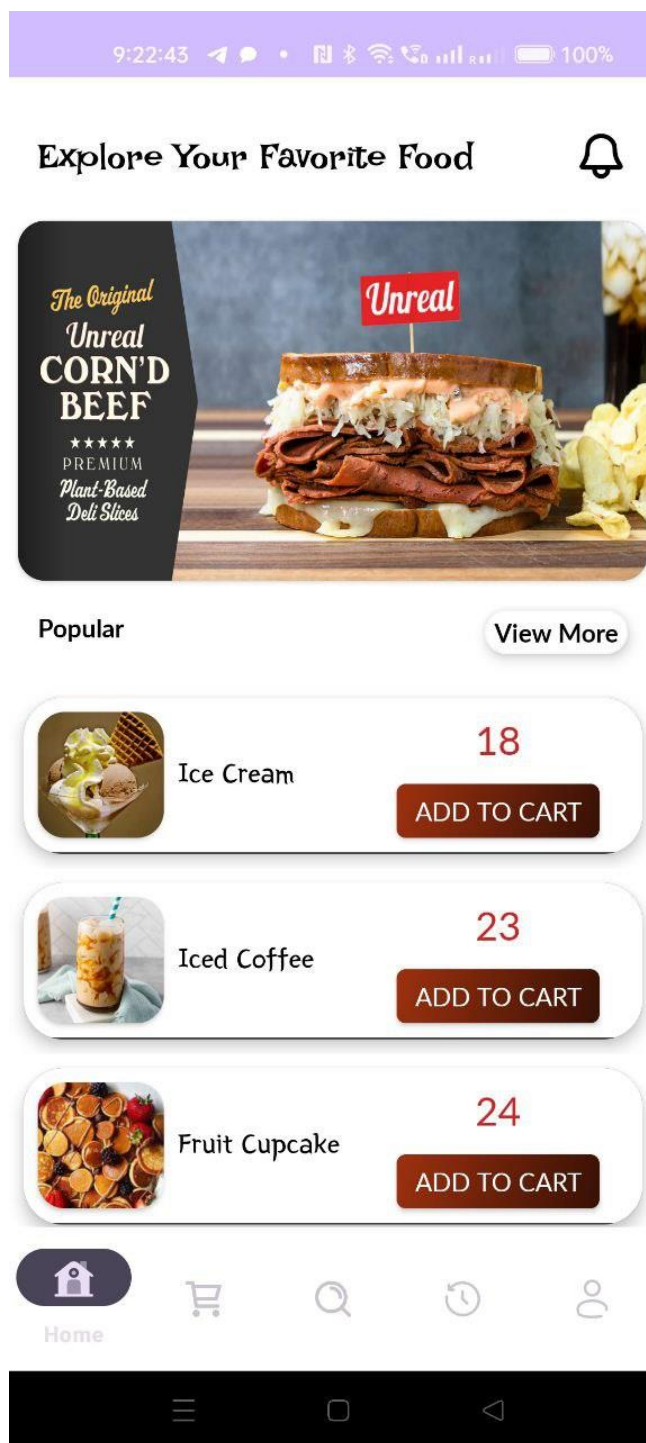
7 Список использованных источников

1. Документация Firebase Realtime Database —
<https://firebase.google.com/docs/database>
<https://console.firebase.google.com/project/yummibox/database/>
2. Документация Firebase Authentication —
<https://firebase.google.com/docs/auth>
<https://console.firebase.google.com/project/yummibox/authentication/users>
3. Документация Supabase Storage —
<https://supabase.com/docs/guides/storage>
<https://supabase.com/dashboard/project/wjejxjzodprauxdjtiys/storage/buckets/images>
4. Документация Glide (Android image loading library) —
<https://bumptech.github.io/glide/>
5. Официальная документация Android Developers —
<https://developer.android.com/>
6. Сайт Figma и гайдлайны по проектированию интерфейсов —
<https://www.figma.com/design/HnqEMkhDBtTw1bp5wyCcQx/YummiBox?node-id=0-1&p=f>
7. Документация по архитектуре MVVM в Android —
<https://developer.android.com/jetpack/guide>
8. GitHub-репозитории проекта:
 - Пользовательское приложение:
<https://github.com/Maryam289/YummiBox>
 - Административная панель: <https://github.com/Maryam289/Admin-YummiBox>

8 Приложения

8.1 Скриншоты интерфейса

Главный экран клиентского приложения;



Страница авторизации/регистрации;

2:22:09

    91%



Sign Up

 Name

 Email

 Password



OR

Sing up With

 GOOGLE

Create Account

Already Have An Account

2:22:04

    91%



Login to your account

 Email

 Password

OR

Continue With

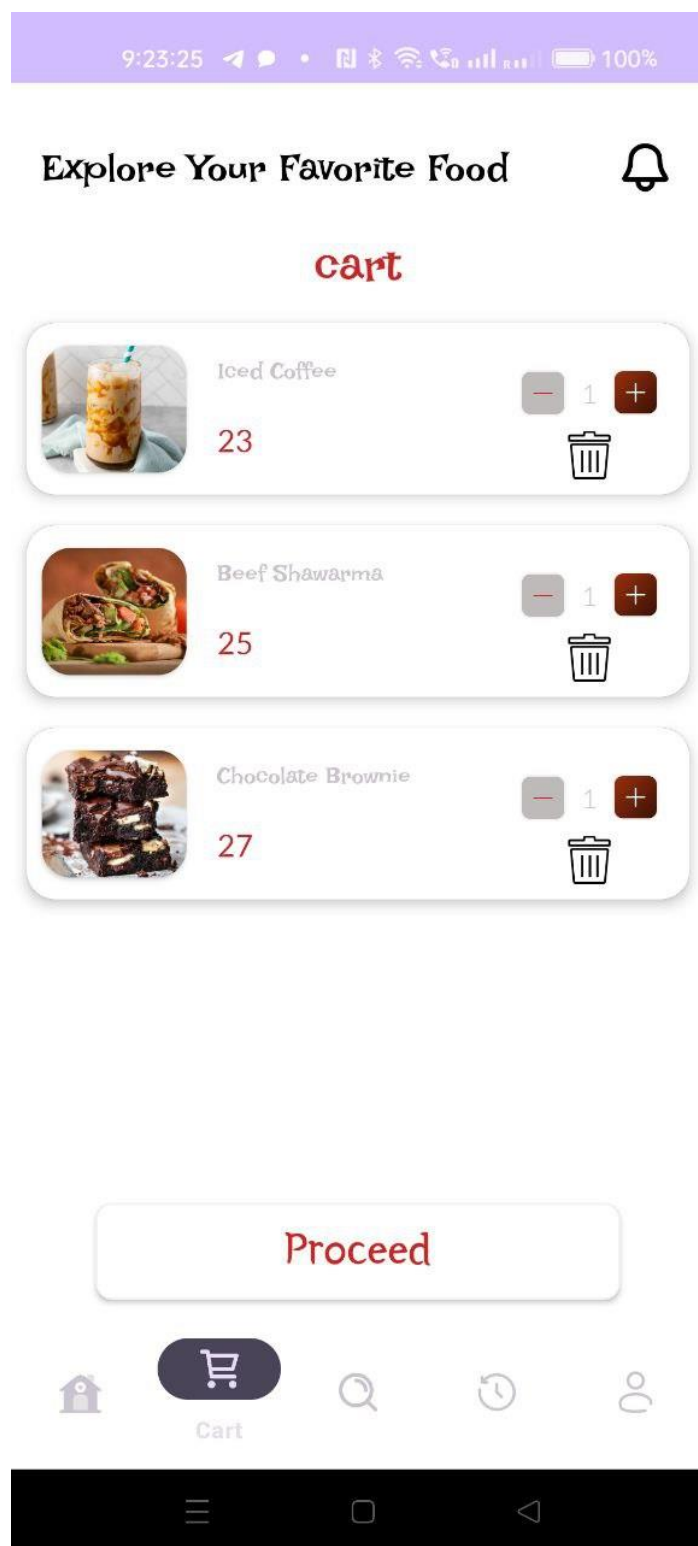
 Google

LOGIN

Don't Have Account

Design By
Maryam

Экран корзины и оформления заказа;



Панель администратора (добавление товара);

9:28:50

100%

←

Add Item

Food Name

Food Price

Select Image

+



Short Description

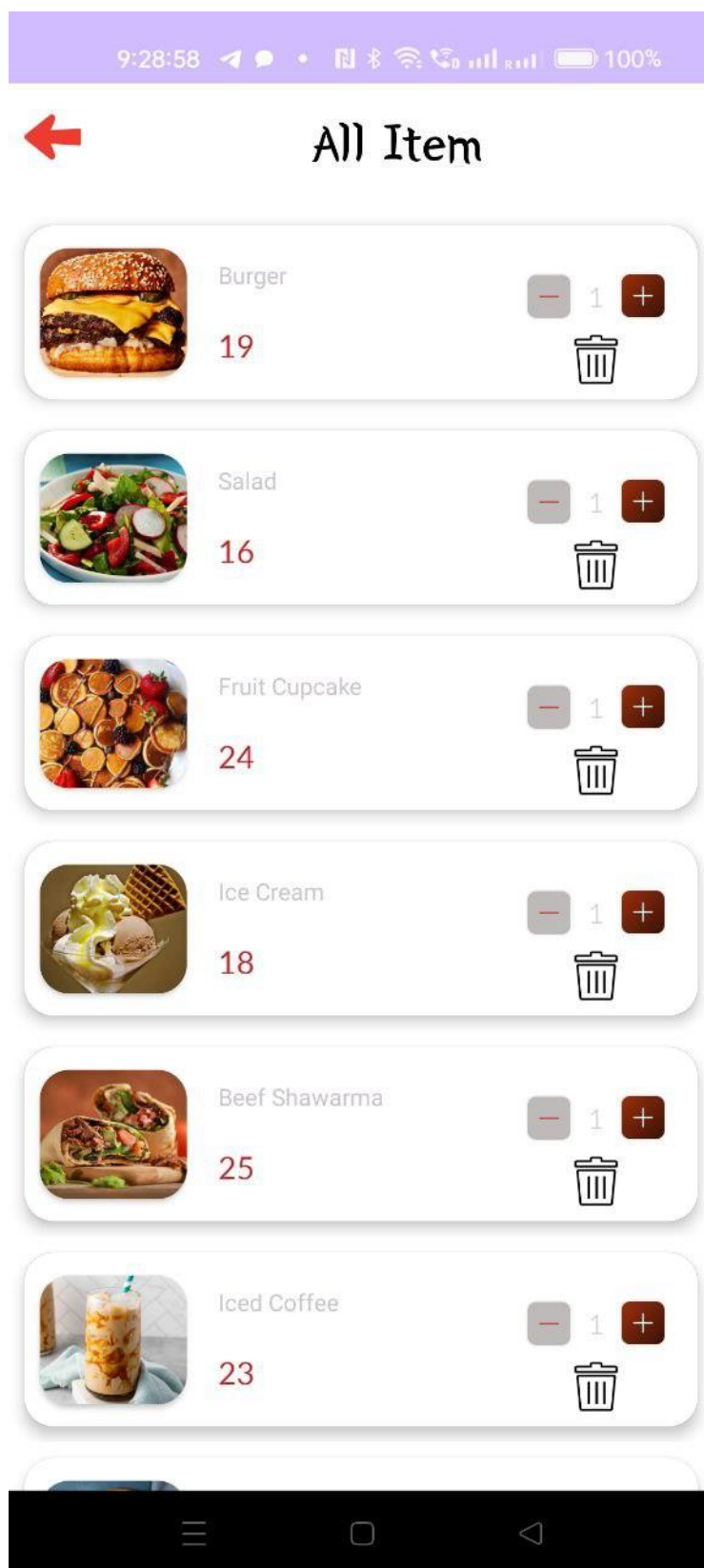
Add Here Description

Ingredients

Add Here Ingradients

Add Item

Интерфейс просмотра заказов;



Пример отображения изображений блюд.

